

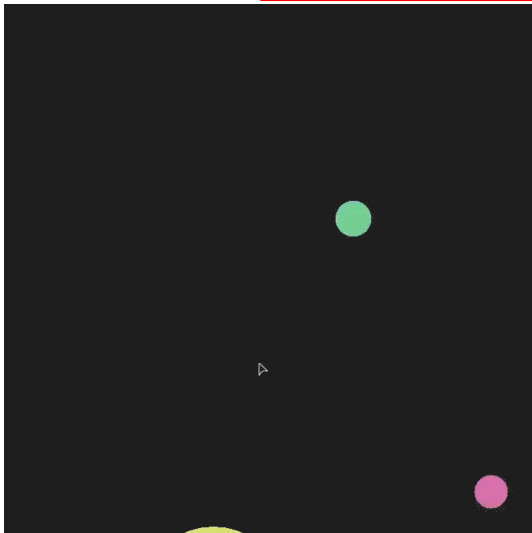


OPENMP - SIMULAÇÃO DE N CORPOS

Lucas Timm

INF01008 - Programação Paralela — Setembro de 2026

- Aluno da Engenharia Civil
- Cursando Extracurricular, não realizei a cadeira de arquitetura ou sistemas previamente
- Já havia realizado este projeto anteriormente, com Raylib
- Leitura prévia de livro sobre OpenMP



1. Duas famílias de algoritmo: Força Bruta e Barnes-Hutt [Chen, 2021].
2. Força bruta : $O(n^2)$
3. Barnes-Hutt : $O(n \log(n))$
4. Força bruta mostra mais potencial para o estudo de paralelização.
5. Cada thread realiza uma quantidade similar de computação.

$$\vec{F} = \frac{\vec{r}}{\|\mathbf{r}\|^3}$$

$$S = S_0 + V_0 t + \frac{at^2}{2}$$

- Ambiente computacional PCAD
- Nó hype5
- CPU: 2 x Intel(R) Xeon(R) E5-2650 v3, 2.3 GHz, 40 threads, 20 cores
- RAM: 128 GB DDR4
- MB: HP ProLiant XL190r Gen9

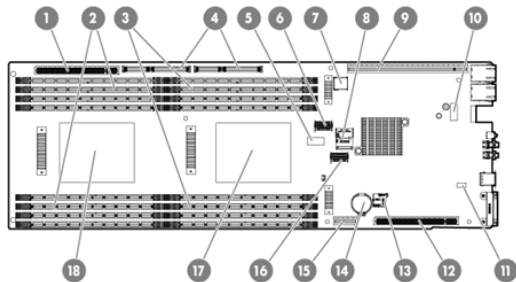


Figura: Esquemática HPE ProLiant XL190r Gen9 Server

1. Implementação em C++
 - -std=c++23 -O3 -march=native -ffast-math -fopenmp
 - Input: .csv com N coordenadas 3d
 - Output:
"S/P;Corpos;Threads;Tempo"
2. Valores de entrada aleatórios, gerados em Python
3. Experimento realizado em script bash, alterando threads e entrada
4. Processamento de resultados em Python

```
16 run_experiment() {  
1  echo "[running] running serial experiment..."  
2  $1 -s $2 >> results.csv  
3  
4  for t in $(seq 2 2 $MAX_THREADS)  
5  do  
6      export OMP_NUM_THREADS=$t  
7      echo "[running] using $OMP_NUM_THREADS threads"  
8      echo "[running] running parallel experiment..."  
9      $1 -p $2 >> results.csv  
10 done  
11 }
```

Figura: Principal função no script bash

Simulação Serial:

1. omp simd
 - Ler corpo i
 - Ler outros corpos, j
 - Calcular participação $\vec{F}_{ij}$
 - Incrementar na força do corpo i
 - Reduzir na força do corpo j
2. omp simd
 - Ler corpo i
 - Atualizar velocidade e posição i

Simulação Paralela:

1. omp for
 - Ler corpo i
 - Ler outros corpos, j
 - Calcular participação $\vec{F}_{ij}$
 - Incrementar na força do corpo i
2. omp barrier
3. omp for simd
 - Ler corpo i
 - Atualizar velocidade e posição i

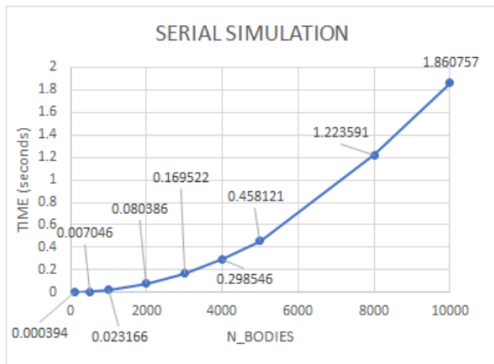


Figure 3: Serial simulation of brute force in one process

Figura: Número de Corpos vs Tempo de Execução, Jiayu Chen 2021

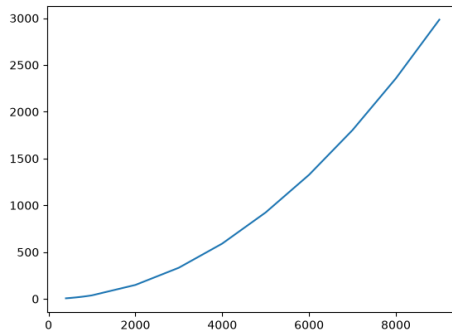
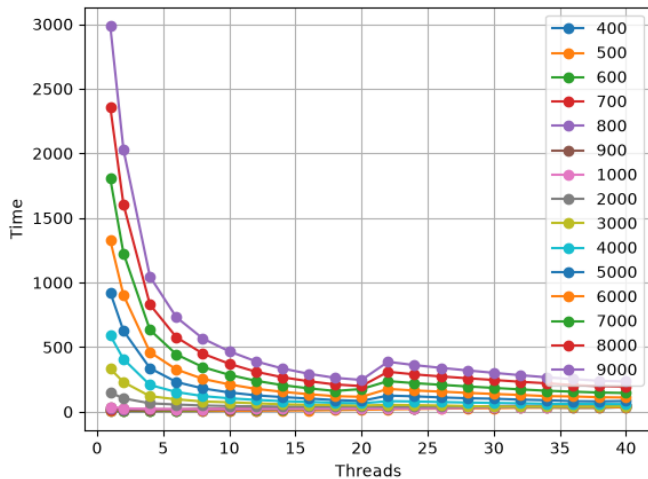
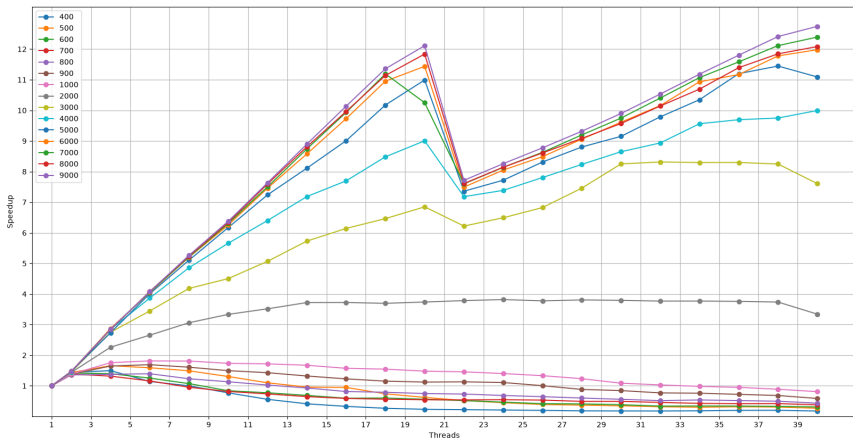
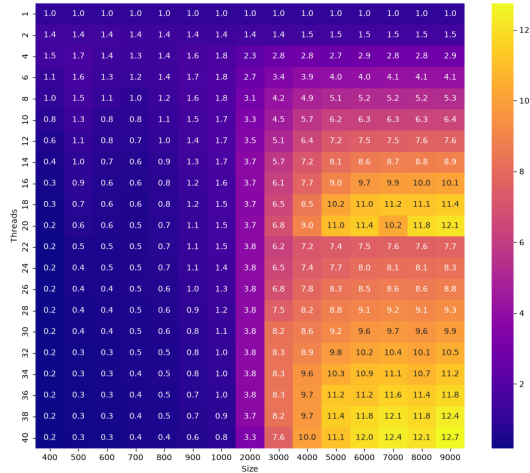
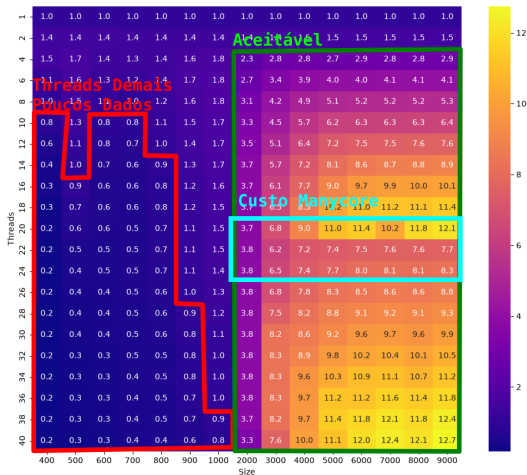


Figura: Número de Corpos vs Tempo de Execução









1. Gargalo de 20-21 threads ocorrido por acessos não uniformes na memória.
2. Latência por acessar a memória próxima do outro processador (NUMA).
3. Ao aumentar o número de threads e problema, deixa de ser tanto problema.
4. Mais threads trabalhando trazem mais dados pra próximo do segundo processador.

- Utilizei o Intel VTune Profile em hpc-performance para testar as hipóteses.
- Retirei 3 casos do mapa de calor : 700 corpos e 5 threads, 4000 corpos e 20 threads, 4000 corpos e 21 threads

20t4000hpc × 21t4000hpc × 5t700hpc × Configure Analysis ×

HPC Performance Characterization HPC Performance Characterization ▾ ⓘ 🔗

Analysis Configuration Collection Log Summary **Bottom-up**

Grouping: Function / Call Stack

Function / Call Stack	CPU Time ⓘ			Instructions Retired	CPI Rate	Average CPU Frequency
	Effective Time ▾	Spin Time	Overhead Time			
► [nbody]	1071.040s	0s	0s	4,477,881,500,000	0.621	2.6 GHz
► [libgomp.so.1.0.0]	53.173s	0s	0s	63,319,000,000	2.178	2.6 GHz
[vmlinux]	2.050s	0s	0s	2,369,000,000	2.112	2.4 GHz

20t4000hpc × 21t4000hpc × 5t700hpc × Configure Analysis ×

HPC Performance Characterization HPC Performance Characterization ▾ ⓘ ⓘ

Analysis Configuration Collection Log Summary Bottom-up libgomp.so.1.0.0 ×

Grouping: Function / Call Stack

Function / Call Stack	CPU Time			Instructions Retired	CPI Rate	Average CPU Frequency
	Effective Time ▾	Spin Time	Overhead Time			
▶ [Loop@0x4e28 in parallel_simulation._omp_fn.0]	1101.485s 	0s	0s	4,478,801,500,000	0.637	2.6 GHz
▶ [Loop@0x1f7d8 in func@0x1f784]	334.373s 	0s	0s	407,537,000,000	2.147	2.6 GHz
▶ [Loop@0x1f620 in func@0x1f5fd]	2.165s	0s	0s	2,254,000,000	2.276	2.4 GHz
▶ [Loop@0x4fd0 in parallel_simulation._omp_fn.0]	1.564s	0s	0s	138,000,000	23.833	2.1 GHz
update_load_sum	0.752s	0s	0s	345,000,000	5.122	2.4 GHz

20t4000hpc × 21t4000hpc × 5t700hpc × Configure Analysis ×

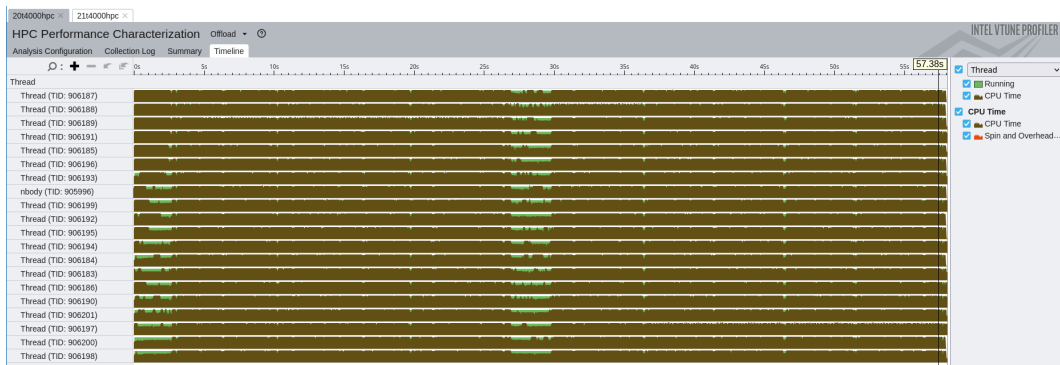
HPC Performance Characterization

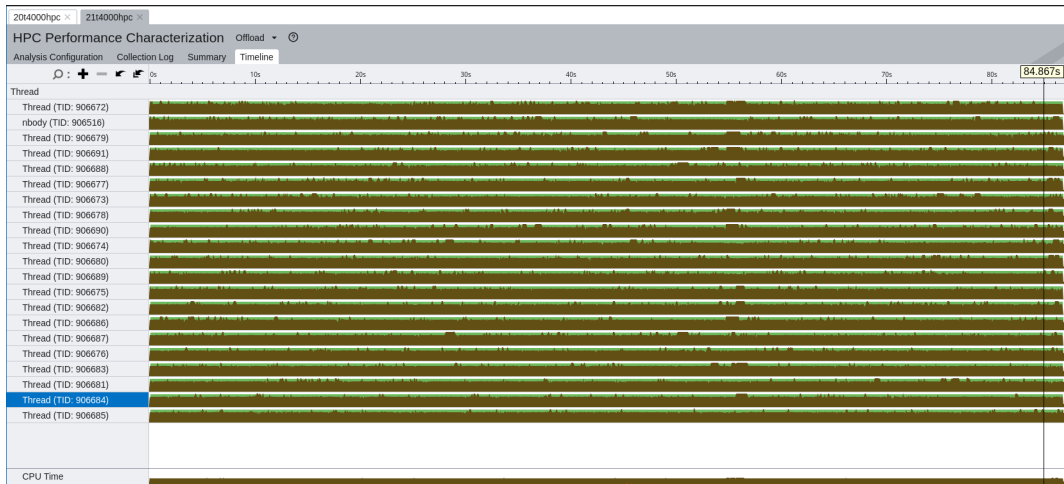
HPC Performance Characterization ▾ ⓘ ⓘ

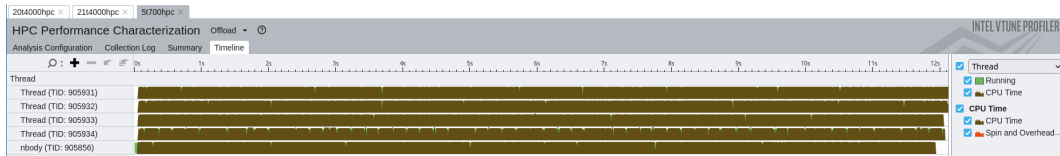
Analysis Configuration Collection Log Summary **Bottom-up**

Grouping: Function / Call Stack ▾ 🔍 🔍 🔍 🔍

Function / Call Stack	CPU Time ⓘ			Instructions Retired	CPI Rate	Average CPU Frequency
	Effective Time ▾	Spin Time	Overhead Time			
▶ [Loop@0x4e28 in parallel_simulation_omp_fn.0]	56.601s	0s	0s	136,275,000,000	1.119	2.7 GHz
▶ [Loop@0x1f7d8 in func@0x1f784]	1.909s	0s	0s	2,553,000,000	2.068	2.8 GHz
▶ [Loop@0x1f620 in func@0x1f5fd]	0.130s	0s	0s	46,000,000	4.250	1.5 GHz
▶ [Loop@0x4fd0 in parallel_simulation_omp_fn.0]	0.125s	0s	0s	11,500,000	35.000	3.2 GHz
▶ func@0x1f934	0.030s	0s	0s	0		0.4 GHz
▶ [Loop@0x1cd60 in func@0x1cc44]	0.015s	0s	0s	0		1.5 GHz







20t4000hpc ×

21t4000hpc ×

5t700hpc ×

Configure Analysis ×

HPC Performance Characterization

Hardware Events ▾ ⓘ ⓘ

Analysis Configuration

Collection Log

Summary

Event Count

Sample Count

Uncore Event Count

Caller/Callee

Top-down Tree

Platform

⌵ Elapsed Time ⓘ: 58.031s >

CPU Time ⓘ: 1126.302s

CPI Rate ⓘ: 0.643

Total Thread Count: 20

Paused Time ⓘ: 0s

⌵ Hardware Events > 📄

Hardware Event Type	Hardware Event Count	Hardware Event Sample Count	Events Per Sample	Precise
CPU_CLK_UNHALTED.ONE_THREAD_ACTIVE	87,537,626,050	25,010	500015	False
CPU_CLK_UNHALTED.REF_TSC	2,584,510,000,000	224,740	11500000	False
CPU_CLK_UNHALTED.REF_XCLK	90,057,701,650	25,730	500015	False
CPU_CLK_UNHALTED.THREAD	2,922,184,500,000	254,103	11500000	False
CYCLE_ACTIVITY.CYCLES_NO_EXECUTE	584,570,876,855	8,351	10000015	False
CYCLE_ACTIVITY.STALLS_L1D_PENDING	224,420,336,630	3,206	10000015	False

20t4000hpc ×
21t4000hpc ×
5t700hpc ×
Configure Analysis ×

HPC Performance Characterization

Hardware Events ⓘ ⓘ

Analysis Configuration
Collection Log
Summary
Event Count
Sample Count
Uncore Event Count
Caller/Callee
Top-down Tree
Platform

⌵ **Elapsed Time ⓘ: 86.816s** > 📄

CPU Time ⓘ: 1445.951s
CPI Rate ⓘ: 0.766
Total Thread Count: 21
Paused Time ⓘ: 0s

⌵ **Hardware Events** >

Hardware Event Type	Hardware Event Count	Hardware Event Sample Count	Events Per Sample	Precise
CPU_CLK_UNHALTED.ONE_THREAD_ACTIVE	98,121,943,570	28,034	500015	False
CPU_CLK_UNHALTED.REF_TSC	3,318,003,000,000	288,522	11500000	False
CPU_CLK_UNHALTED.REF_XCLK	112,664,879,845	32,189	500015	False
CPU_CLK_UNHALTED.THREAD	3,751,369,000,000	326,206	11500000	False
CYCLE_ACTIVITY.CYCLES_NO_EXECUTE	976,291,464,435	13,947	10000015	False
CYCLE_ACTIVITY.STALLS_L1D_PENDING	98,210,147,315	1,403	10000015	False

20t4000hpc × 21t4000hpc × 5t700hpc × Configure Analysis ×

HPC Performance Characterization

Hardware Events ▾ ? ⓘ

Analysis Configuration

Collection Log

Summary

Event Count

Sample Count

Uncore Event Count

Caller/Callee

Top-down Tree

Platform

⌵ Elapsed Time ⓘ: 12.122s >

CPU Time ⓘ: 58.996s

CPI Rate ⓘ: 1.142 ⚡

Total Thread Count: 5

Paused Time ⓘ: 0s

⌵ Hardware Events > 📄

Hardware Event Type	Hardware Event Count	Hardware Event Sample Count	Events Per Sample	Precise
CPU_CLK_UNHALTED.ONE_THREAD_ACTIVE	4,756,642,695	1,359	500015	False
CPU_CLK_UNHALTED.REF_TSC	135,378,000,000	11,772	11500000	False
CPU_CLK_UNHALTED.REF_XCLK	4,749,642,485	1,357	500015	False
CPU_CLK_UNHALTED.THREAD	158,964,500,000	13,823	11500000	False
CYCLE_ACTIVITY.CYCLES_NO_EXECUTE	83,510,125,265	1,193	10000015	False
CYCLE_ACTIVITY.STALLS_L1D_PENDING	52,710,079,065	753	10000015	False
CYCLE_ACTIVITY.STALLS_L2_PENDING	73,780,110,670	1,054	10000015	False
CYCLE_ACTIVITY.STALLS_LDM_PENDING	74,270,111,405	1,061	10000015	False
IDQ_UOPS_NOT_DELIVERED.CORE	28,070,042,105	401	10000015	False
IDQ_UOPS_NOT_DELIVERED.CYCLES_0_UOPS_DELIV.CORE	2,170,003,255	31	10000015	False
INST_RETIRED_ANY	133,313,333,333	13,333	11500000	False

- A noção que passar para 21 threads traz latência é colaborada pelos eventos.
- CPI maior na run de 21 threads demonstra que há mais latência comparado com 20.
- Entretanto, a presença de mais números do evento CPU UNHALTED ONE THREAD ACTIVE com 21 threads, traz o hyperthreading como suspeito.
- A run de 21 threads também fica bastante tempo parada em uma call do OMP (libgomp), comparada com a de 20 threads.
- Evidencia que possa ser não o acesso à memória, mas talvez um núcleo em hyperthreading, atrasando todos os outros núcleos na chamada de barreira.
- Um thread lerdo, atrasando os outros.
- Explicaria o Speedup retornar a subir quando aumentamos o número de threads, mais delas estão em hyperthreading, mais threads fazendo trabalho similar, diminuindo o peso da barreira.

- CHEN, Jiayu. Understanding and Analyzing the Performance Scalability of N-body Problem. ACM, 2021. Disponível em: <https://dl.acm.org/doi/abs/10.1145/3494885.3494924>. Acesso em: 28 set. 2026.
- SILVA, Gabriel P.; BIANCHINI, Calebe P.; COSTA, Evaldo B.. Programação Paralela e Distribuída: com MPI, OpenMP e OpenACC para computação de alto desempenho. São Paulo: Casa do Código, 2022.

Lucas Timm

Instituto de Informática | Escola de Engenharia — UFRGS

